

LAPORAN PRAKTIKUM
ALGORITMA PEMROGRAMAN 2
MODUL 14 – SORTING



NAMA : Yazid Yaskur Muhammad

NIM : 109082500078

KELAS S1IF-13-06

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Algoritma sorting adalah teknik yang digunakan untuk mengurutkan data berdasarkan kriteria tertentu, seperti urutan menaik (*ascending*) atau menurun (*descending*). Proses pengurutan sangat penting dalam pemrograman karena dapat mempermudah pencarian data, pengelolaan informasi, serta meningkatkan efisiensi berbagai algoritma lainnya. Pada modul ini dibahas dua metode pengurutan sederhana, yaitu Selection Sort dan Insertion Sort, yang bekerja dengan cara berbeda namun memiliki tujuan yang sama, yaitu menghasilkan data yang terurut.

Algoritma sorting adalah teknik yang digunakan untuk mengurutkan data berdasarkan kriteria tertentu, seperti urutan menaik (*ascending*) atau menurun (*descending*). Proses pengurutan sangat penting dalam pemrograman karena dapat mempermudah pencarian data, pengelolaan informasi, serta meningkatkan efisiensi berbagai algoritma lainnya. Pada modul ini dibahas dua metode pengurutan sederhana, yaitu Selection Sort dan Insertion Sort, yang bekerja dengan cara berbeda namun memiliki tujuan yang sama, yaitu menghasilkan data yang terurut.

B. Guided 1

```
package main

import "fmt"

func selectionSortArray(angka *[5]int) {
    var idxMin, i, j int

    for i = 0; i < len(angka)-1; i++ { // loop luar (iterasi sorting)
        idxMin = i

        for j = i + 1; j < len(angka); j++ { // loop dalam (mencari nilai minimum)
            if angka[j] <= angka[idxMin] { // ascending
                idxMin = j
            }
        }

        if idxMin != i {
            angka[i], angka[idxMin] = angka[idxMin],
angka[i]
        }
    }
}
```

```

func main() {
    var arrAngka [5]int

    for i := 0; i < len(arrAngka); i++ {
        fmt.Printf("Masukkan data angka ke-%d : ", i+1)
        fmt.Scan(&arrAngka[i])
    }

    fmt.Println()
    fmt.Println("=== SEBELUM DISORTING ===")
    for i := 0; i < len(arrAngka); i++ {
        fmt.Print(arrAngka[i], " ")
    }

    fmt.Println()
    fmt.Println("=== SETELAH DISORTING ===")

    selectionSortArray(&arrAngka)

    for i := 0; i < len(arrAngka); i++ {
        fmt.Print(arrAngka[i], " ")
    }

    fmt.Println()
}

```

Output :

```

PS C:\Users\Asus\Documents\AlproPraktikum> go run
ded1\n01.go
Masukkan data angka ke-1 : 5
Masukkan data angka ke-2 : 4
Masukkan data angka ke-3 : 3
Masukkan data angka ke-4 : 2
Masukkan data angka ke-5 : 1

=== SEBELUM DISORTING ===
5 4 3 2 1
=== SETELAH DISORTING ===
1 2 3 4 5

```

2. Guided 2

```
package main

import "fmt"

const nMax int = 4321

type arrInt [nMax]int

func insertionSort(T *arrInt, n int) {
    /* I.S. terdefinisi array T yang berisi n bilangan bulat
       F.S. array T terurut secara mengecil (descending)
       dengan INSERTION SORT
    */

    var temp, i, j int

    i = 1
    for i <= n-1 {
        j = i
        temp = T[j]

        for j > 0 && temp > T[j-1] {
            T[j] = T[j-1]
            j = j - 1
        }

        T[j] = temp
        i = i + 1
    }
}

func main() {
    var data arrInt
    var n int

    fmt.Print("Masukkan jumlah data: ")
    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        fmt.Printf("Data ke-%d: ", i+1)
        fmt.Scan(&data[i])
    }
}
```

```
    fmt.Println("\nSebelum sorting:")
    for i := 0; i < n; i++ {
        fmt.Print(data[i], " ")
    }

    insertionSort(&data, n)

    fmt.Println("\n\nSetelah sorting (descending):")
    for i := 0; i < n; i++ {
        fmt.Print(data[i], " ")
    }
    fmt.Println()
}
```

```

PS C:\Users\Asus\Documents\AlproPraktikum> go run "c:\Users\Asus\Documents\AlproPraktikum\ded2\no2.go"
Masukkan jumlah data: 5
Data ke-1: 7 2 9 4 1
Data ke-2: Data ke-3: Data ke-4: Data ke-5:
Sebelum sorting:
7 2 9 4 1

Setelah sorting (descending):
9 7 4 2 1
PS C:\Users\Asus\Documents\AlproPraktikum> go run "c:\Users\Asus\Documents\AlproPraktikum\ded2\no2.go"
Masukkan jumlah data: 6
Data ke-1: 10 3 15 8 6 20
Data ke-2: Data ke-3: Data ke-4: Data ke-5: Data ke-6:
Sebelum sorting:
10 3 15 8 6 20

Setelah sorting (descending):
20 15 10 8 6 3

```

3. Guided 3

```

package main

import "fmt"

const nMax int = 100

type arrString [nMax]string

func insertionSortString(T *arrString, n int) {
    var temp string
    var i, j int

    i = 1
    for i <= n-1 {
        j = i
        temp = T[j]

        // Ascending (A-Z)
        for j > 0 && temp < T[j-1] {
            T[j] = T[j-1]
            j = j - 1
        }

        T[j] = temp
        i = i + 1
    }
}

```

```

    }
}

func main() {
    var data arrString
    var n int

    fmt.Print("Masukkan jumlah data: ")
    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        fmt.Printf("Data ke-%d: ", i+1)
        fmt.Scan(&data[i])
    }

    fmt.Println("\nSebelum sorting:")
    for i := 0; i < n; i++ {
        fmt.Print(data[i], " ")
    }

    insertionSortString(&data, n)

    fmt.Println("\n\nSetelah sorting (ascending):")
    for i := 0; i < n; i++ {
        fmt.Print(data[i], " ")
    }
    fmt.Println()
}

```

Output:

```

PS C:\Users\Asus\Documents\AlproPraktikum> go run "c:\Users\Asus\
ded3\no3.go"
Masukkan jumlah data: 5
Data ke-1: apel
Data ke-2: jeruk
Data ke-3: mangga
Data ke-4: pisang
Data ke-5: anggur

Sebelum sorting:
apel jeruk mangga pisang anggur

Setelah sorting (ascending):
anggur apel jeruk mangga pisang
PS C:\Users\Asus\Documents\AlproPraktikum> go run "c:\Users\Asus\
ded3\no3.go"
Masukkan jumlah data: 4
Data ke-1: kuda
Data ke-2: sapi
Data ke-3: zebra
Data ke-4: burung

Sebelum sorting:
kuda sapi zebra burung

Setelah sorting (ascending):
burung kuda sapi zebra

```

4. Guided 4

```
package main

import "fmt"

type mahasiswa struct {
    nama string
    nim  string
    ipk  float64
}

func selectionSort(mahasiswaList []mahasiswa) {
    n := len(mahasiswaList)

    for i := 0; i < n-1; i++ {
        idxMin := i

        for j := i + 1; j < n; j++ {
            if mahasiswaList[j].ipk <
mahasiswaList[idxMin].ipk {
                idxMin = j
            }
        }

        if idxMin != i {
            mahasiswaList[i], mahasiswaList[idxMin] =
                mahasiswaList[idxMin], mahasiswaList[i]
        }
    }
}

func printData(mahasiswaList []mahasiswa, status string) {
    fmt.Printf("\n=== %s ===\n", status)

    for i, mhs := range mahasiswaList {
        fmt.Printf("\n--- Data Mahasiswa Ke-%d ---\n",
i+1)

        fmt.Printf("Nama : %s\n", mhs.nama)
        fmt.Printf("NIM  : %s\n", mhs.nim)
        fmt.Printf("IPK   : %.2f\n", mhs.ipk)
    }

    fmt.Println("=====")
```



```
}

func main() {
    structMHS := make([]mahasiswa, 5)

    for i := 0; i < len(structMHS); i++ {
        fmt.Printf("\n--- Masukkan Data Mahasiswa Ke-%d --
\n", i+1)

        fmt.Print("Nama : ")
        fmt.Scan(&structMHS[i].nama)

        fmt.Print("NIM   : ")
        fmt.Scan(&structMHS[i].nim)

        fmt.Print("IPK   : ")
        fmt.Scan(&structMHS[i].ipk)
    }

    printData(structMHS, "SEBELUM SORTING")

    selectionSort(structMHS)

    printData(structMHS, "SETELAH SORTING")
}
```

Output:

```
--- Masukkan Data Mahasiswa Ke-1 ---
Nama : Yaskur
NIM : 1078
IPK : 4.0

--- Masukkan Data Mahasiswa Ke-2 ---
Nama : Erpan
NIM : 1079
IPK : 3.9

--- Masukkan Data Mahasiswa Ke-3 ---
Nama : Hanif
NIM : 1080
IPK : 3.8

--- Masukkan Data Mahasiswa Ke-4 ---
Nama : Arsyia
NIM : 1081
IPK : 3.7

--- Masukkan Data Mahasiswa Ke-5 ---
Nama : Faiq
NIM : 1082
IPK : 3.6

=== SEBELUM SORTING ===

--- Data Mahasiswa Ke-1 ---
Nama : Yaskur
NIM : 1078
IPK : 4.00

--- Data Mahasiswa Ke-2 ---
Nama : Erpan
NIM : 1079
IPK : 3.90

--- Data Mahasiswa Ke-3 ---
Nama : Hanif
NIM : 1080
IPK : 3.80

--- Data Mahasiswa Ke-4 ---
Nama : Arsyia
NIM : 1081
IPK : 3.70

--- Data Mahasiswa Ke-5 ---
Nama : Faiq
NIM : 1082
IPK : 3.60
=====
```

=== SETELAH SORTING ===

```
--- Data Mahasiswa Ke-1 ---
Nama : Faiq
NIM : 1082
IPK : 3.60

--- Data Mahasiswa Ke-2 ---
Nama : Arsyia
NIM : 1081
IPK : 3.70

--- Data Mahasiswa Ke-3 ---
Nama : Hanif
NIM : 1080
IPK : 3.80

--- Data Mahasiswa Ke-4 ---
Nama : Erpan
NIM : 1079
IPK : 3.90

--- Data Mahasiswa Ke-5 ---
Nama : Yaskur
NIM : 1078
IPK : 4.00

=====
```

C. Unguided

1. Hercules, preman terkenal seantero ibukota, memiliki kerabat di banyak daerah. Tentunya Hercules sangat suka mengunjungi semua kerabatnya itu. Diberikan masukan nomor rumah dari semua kerabatnya di suatu daerah, buatlah program rumahkerabat yang akan menyusun nomor-nomor rumah kerabatnya secara terurut membesar menggunakan algoritma selection sort.

Masukan dimulai dengan sebuah integer n ($0 < n < 1000$), banyaknya daerah kerabat Hercules tinggal. Isi n baris berikutnya selalu dimulai dengan sebuah integer m ($0 < m < 1000000$) yang menyatakan banyaknya rumah kerabat di daerah tersebut, diikuti dengan rangkaian bilangan bulat positif, nomor rumah para kerabat.

Keluaran terdiri dari n baris, yaitu rangkaian rumah kerabatnya terurut membesar di masingmasing daerah.

Jawaban:

```
package main

import (
    "bufio"
    "fmt"
    "os"
)

func selectionSort(arr []int) {
    n := len(arr)

    for i := 0; i < n-1; i++ {
        idxMin := i

        for j := i + 1; j < n; j++ {
            if arr[j] < arr[idxMin] {
                idxMin = j
            }
        }

        if idxMin != i {
            arr[i], arr[idxMin] = arr[idxMin], arr[i]
        }
    }
}

func main() {
    reader := bufio.NewReader(os.Stdin)
    writer := bufio.NewWriter(os.Stdout)
    defer writer.Flush()
}
```

```

var n int

_, err := fmt.Fscan(reader, &n)
if err != nil || n <= 0 || n >= 1000 {
    return
}

for i := 0; i < n; i++ {
    var m int
    fmt.Fscan(reader, &m)

    rumah := make([]int, m)

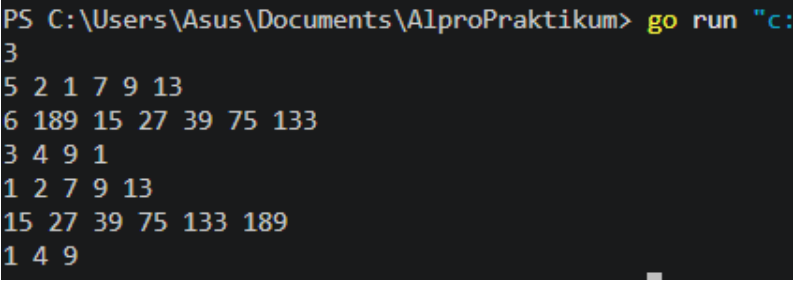
    for j := 0; j < m; j++ {
        fmt.Fscan(reader, &rumah[j])
    }

    selectionSort(rumah)

    for j, nilai := range rumah {
        if j > 0 {
            fmt.Fprint(writer, " ")
        }
        fmt.Fprint(writer, nilai)
    }
    fmt.Fprintln(writer)
}
}

```

Output:



```

PS C:\Users\Asus\Documents\AlproPraktikum> go run "c:
3
5 2 1 7 9 13
6 189 15 27 39 75 133
3 4 9 1
1 2 7 9 13
15 27 39 75 133 189
1 4 9

```

Penjelasan:

Program tersebut bekerja dengan cara membaca sejumlah data dari input, lalu mengurutkan setiap kumpulan angka menggunakan algoritma Selection Sort. Pertama fungsi selectionSort akan mencari elemen terkecil dari bagian array yang belum terurut, kemudian menukarnya dengan elemen di posisi saat ini. Proses ini diulang sampai seluruh array terurut secara ascending.

2. Belakangan diketahui ternyata Hercules itu tidak berani menyeberang jalan, maka selalu diusahakan agar hanya menyeberang jalan sesedikit mungkin, hanya diujung jalan. Karena nomor rumah sisi kiri jalan selalu ganjil dan sisi kanan jalan selalu genap, maka buatlah program kerabat dekat yang akan menampilkan nomor rumah mulai dari nomor yang ganjil lebih dulu terurut membesar dan kemudian menampilkan nomor rumah dengan nomor genap terurut mengecil.

Format **Masukan** masih persis sama seperti sebelumnya.

Keluaran terdiri dari n baris, yaitu rangkaian rumah kerabatnya terurut membesar untuk nomor ganjil, diikuti dengan terurut mengecil untuk nomor genap, di masing-masing daerah.

Jawaban:

```
package main

import (
    "bufio"
    "fmt"
    "os"
)

func selectionSortAsc(arr []int) {
    n := len(arr)

    for i := 0; i < n-1; i++ {
        idxMin := i

        for j := i + 1; j < n; j++ {
            if arr[j] < arr[idxMin] {
                idxMin = j
            }
        }

        if idxMin != i {
            arr[i], arr[idxMin] =
arr[idxMin], arr[i]
        }
    }
}

func selectionSortDesc(arr []int) {
    n := len(arr)

    for i := 0; i < n-1; i++ {
```

```

idxMax := i

for j := i + 1; j < n; j++ {
    if arr[j] > arr[idxMax] {
        idxMax = j
    }
}

if idxMax != i {
    arr[i], arr[idxMax] =
arr[idxMax], arr[i]
}

}

func main() {

    reader := bufio.NewReader(os.Stdin)
    writer := bufio.NewWriter(os.Stdout)
    defer writer.Flush()

    var n int

    _, err := fmt.Fscan(reader, &n)
    if err != nil || n <= 0 || n >= 1000 {
        return
    }

    for i := 0; i < n; i++ {
        var m int
        fmt.Fscan(reader, &m)

        var ganjil []int
        var genap []int

        for j := 0; j < m; j++ {
            var nilai int
            fmt.Fscan(reader, &nilai)

            if nilai%2 == 0 {
                genap = append(genap,
nilai)
            } else {

```

```

                                ganjil = append(ganjil,
nilai)
                                }
                                }

                                selectionSortAsc(ganjil)
                                selectionSortDesc(genap)

                                isFirst := true

                                for _, nilai := range ganjil {
                                    if !isFirst {
                                        fmt.Fprint(writer, " ")
                                    }
                                    fmt.Fprint(writer, nilai)
                                    isFirst = false
                                }

                                for _, nilai := range genap {
                                    if !isFirst {
                                        fmt.Fprint(writer, " ")
                                    }
                                    fmt.Fprint(writer, nilai)
                                    isFirst = false
                                }

                                fmt.Fprintln(writer)
                                }
}

```

Output:

```

PS C:\Users\Asus\Documents\AlproPraktikum> go run "c:\User
3
5 2 1 7 9 13
6 189 15 27 39 75 133
3 4 9 1
1 7 9 13 2
15 27 39 75 133 189
1 9 4

```

Penjelasan:

Program ini membaca sejumlah kumpulan angka, lalu memisahkan setiap angka menjadi dua kelompok; ganjil dan genap. Angka-angka ganjil kemudian diurutkan secara ascending (dari

kecil ke besar) menggunakan fungsi `selectionSortAsc`, sedangkan angka-angka genap diurutkan secara descending (dari besar ke kecil) menggunakan fungsi `selectionSortDesc`.

3. Buatlah sebuah program yang digunakan untuk membaca data integer seperti contoh yang diberikan di bawah ini, kemudian diurutkan (menggunakan metoda insertion sort), dan memeriksa apakah data yang terurut berjarak sama terhadap data sebelumnya.

Masukan terdiri dari sekumpulan bilangan bulat yang diakhiri oleh bilangan negatif. Hanya bilangan non negatif saja yang disimpan ke dalam array.

Keluaran terdiri dari dua baris. Baris pertama adalah isi dari array setelah dilakukan pengurutan, sedangkan baris kedua adalah status jarak setiap bilangan yang ada di dalam array. "Data berjarak x" atau "data berjarak tidak tetap".

Jawaban:

```
package main

import "fmt"

func main() {
    var input int
    var data []int

    for {
        _, err := fmt.Scan(&input)

        if err != nil {
            break
        }

        if input < 0 {
            break
        }

        data = append(data, input)
    }

    if len(data) == 0 {
        return
    }

    insertionSort(data)

    for i, val := range data {
        if i > 0 {
```



```

        fmt.Print(" ")
    }
    fmt.Print(val)
}
fmt.Println()

statusJarak(data)
}

func insertionSort(arr []int) {
    n := len(arr)

    for i := 1; i < n; i++ {
        key := arr[i]
        j := i - 1

        for j >= 0 && arr[j] > key {
            arr[j+1] = arr[j]
            j--
        }

        arr[j+1] = key
    }
}

func statusJarak(arr []int) {
    n := len(arr)

    if n <= 1 {
        fmt.Println("Data berjarak 0")
        return
    }

    jarakAwal := arr[1] - arr[0]
    isiTetap := true

    for i := 1; i < n-1; i++ {
        if arr[i+1]-arr[i] != jarakAwal {
            isiTetap = false
            break
        }
    }

    if isiTetap {
        fmt.Printf("Data berjarak %d\n", jarakAwal)
    } else {

```

```

        fmt.Println("Data berjarak tidak tetap")
    }
}

```

Output:

```

PS C:\Users\Asus\Documents\AlproPraktikum> go run '
nguided3\n03.go'
31 13 25 43 1 7 19 37 -5
1 7 13 19 25 31 37 43
Data berjarak 6
PS C:\Users\Asus\Documents\AlproPraktikum> go run '
nguided3\n03.go'
4 40 14 8 26 1 38 2 32 -31
1 2 4 8 14 26 32 38 40
Data berjarak tidak tetap

```

Penjelasan:

Program bekerja dengan cara membaca sejumlah bilangan dari input, lalu berhenti jika menemukan angka negatif atau jika input tidak valid. Semua bilangan yang valid disimpan dalam slice data. Setelah selesai membaca, program akan mengurutkan bilangan tersebut menggunakan algoritma insertion sort, yaitu dengan cara menyisipkan setiap elemen ke posisi yang tetap sehingga seluruh data terurut naik.

- Sebuah program perpustakaan digunakan untuk mengelola data buku di dalam suatu perpustakaan. Misalnya terdefinisi struct dan array seperti berikut ini:

```

const nMax : integer = 7919
type Buku = <
    id, judul, penulis, penerbit : string
    eksemplar, tahun, rating : integer >
type DaftarBuku = array [ 1..nMax] of Buku
Pustaka : DaftarBuku
nPustaka: integer

```

Masukan terdiri dari beberapa baris. Baris pertama adalah bilangan bulat N yang menyatakan banyaknya data buku yang ada di dalam perpustakaan. N baris berikutnya, masing-masingnya adalah data buku sesuai dengan atribut atau field pada struct. Baris terakhir adalah bilangan bulat yang menyatakan rating buku yang akan dicari

Keluaran terdiri dari beberapa baris. Baris pertama adalah data buku terfavorit, baris kedua adalah lima judul buku dengan rating tertinggi, selanjutnya baris terakhir adalah data buku yang dicari sesuai rating yang diberikan pada masukan baris terakhir.

Jawaban:

```

package main

import (
    "bufio"
    "fmt"
    "os"
    "strconv"
    "strings"
)

const nMax = 7919

type Buku struct {
    id, judul, penulis, penerbit string
    eksemplar, tahun, rating      int
}

type DaftarBuku [nMax]Buku

func main() {
    var pustaka DaftarBuku
    var nPustaka int
    var ratingDicari int

    reader := bufio.NewReader(os.Stdin)

    fmt.Scanln(&nPustaka)

    BacaDaftarBuku(&pustaka, nPustaka, reader)

    inputRating, _ := reader.ReadString('\n')
    inputRating = strings.TrimSpace(inputRating)
    ratingDicari, _ = strconv.Atoi(inputRating)

    CetakTerfavorit(pustaka, nPustaka)

    UrutBuku(&pustaka, nPustaka)

    Cetak5Terbaru(pustaka, nPustaka)

    CariBuku(pustaka, nPustaka, ratingDicari)
}

func BacaDaftarBuku(pustaka *DaftarBuku, n int, reader
*bufio.Reader) {
    for i := 0; i < n; i++ {

```

```

var b Buku

b.id, _ = reader.ReadString('\n')
b.id = strings.TrimSpace(b.id)

b.judul, _ = reader.ReadString('\n')
b.judul = strings.TrimSpace(b.judul)

b.penulis, _ = reader.ReadString('\n')
b.penulis = strings.TrimSpace(b.penulis)

b.penerbit, _ = reader.ReadString('\n')
b.penerbit = strings.TrimSpace(b.penerbit)

txtEksemplar, _ := reader.ReadString('\n')
b.eksemplar, _ =
strconv.Atoi(strings.TrimSpace(txtEksemplar))

txtTahun, _ := reader.ReadString('\n')
b.tahun, _ = strconv.Atoi(strings.TrimSpace(txtTahun))

txtRating, _ := reader.ReadString('\n')
b.rating, _ =
strconv.Atoi(strings.TrimSpace(txtRating))

pustaka[i] = b
}
}

func CetakTerfavorit(pustaka DaftarBuku, n int) {
    if n == 0 {
        return
    }

    maxIdx := 0

    for i := 1; i < n; i++ {
        if pustaka[i].rating > pustaka[maxIdx].rating {
            maxIdx = i
        }
    }

    fav := pustaka[maxIdx]

    fmt.Printf("%s, %s, %s, %d\n",
        fav.judul,

```

```

        fav.penulis,
        fav.penerbit,
        fav.tahun)
    }

func UrutBuku(pustaka *DaftarBuku, n int) {
    for i := 1; i < n; i++ {
        key := pustaka[i]
        j := i - 1

        for j >= 0 && pustaka[j].rating < key.rating {
            pustaka[j+1] = pustaka[j]
            j--
        }

        pustaka[j+1] = key
    }
}

func Cetak5Terbaru(pustaka DaftarBuku, n int) {
    limit := 5

    if n < 5 {
        limit = n
    }

    var judulList []string

    for i := 0; i < limit; i++ {
        judulList = append(judulList, pustaka[i].judul)
    }

    fmt.Println(strings.Join(judulList, " "))
}

func CariBuku(pustaka DaftarBuku, n int, r int) {
    left := 0
    right := n - 1
    foundIdx := -1

    for left <= right {
        mid := left + (right-left)/2

        if pustaka[mid].rating == r {
            foundIdx = mid
            break
        }
    }
}

```

```

        } else if pustaka[mid].rating < r {
            right = mid - 1
        } else {
            left = mid + 1
        }
    }

    if foundIdx != -1 {
        b := pustaka[foundIdx]

        fmt.Printf("%s, %s, %s, %d, %d, %d\n",
            b.judul,
            b.penulis,
            b.penerbit,
            b.tahun,
            b.eksemplar,
            b.rating)
    } else {
        fmt.Println("Tidak ada buku dengan rating seperti itu")
    }
}

```

Output:

```

PS C:\Users\Asus\Documents\AlproPraktikum> go run "c:\Users\Asus\
nguided4\no4.go"
3
B01
Laskar Pelangi
Andrea Hirata
Bentang Pustaka
10
2005
92
B02
Bumi Manusia
Pramoedya Ananta Toer
Lentera Dipantara
5
1980
95
B03
Filosofi Kopi
Dee Lestari
Truedee Books
15
2006
88
92
Bumi Manusia, Pramoedya Ananta Toer, Lentera Dipantara, 1980
Bumi Manusia Laskar Pelangi Filosofi Kopi
Laskar Pelangi, Andrea Hirata, Bentang Pustaka, 2005, 10, 92

```

Penjelasan:

Program tersebut adalah sistem sederhana untuk mengelola data buku dalam sebuah pustaka. Pertama program mendefinisikan tipe data Buku yang berisi informasi seperti id, judul, penulis, penerbit, jumlah eksemplar, tahun terbit, dan rating. Semua buku disimpan dalam array bernama DaftarBuku. Pada fungsi main, pengguna diminta memasukkan jumlah buku yang akan dicatat (nPustaka)., kemudian setiap detail buku dibaca satu per satu melalui fungsi BacaDaftarBuku. Setelah semua data masuk, program juga membaca sebuah nilai rating yang akan digunakan untuk pencarian.

D.

Kesimpulan

Kesimpulan dari praktikum pada modul tersebut bahwa algoritma sorting seperti selection sort dan insertion sort digunakan untuk mengurutkan data dengan cara yang berbeda, namun sama-sama bertujuan menghasilkan data yang terstruktur rapi. Selection sort bekerja dengan cara yang berbeda dengan mencari nilai ekstrem (terkecil atau terbesar) lalu menukarnya ke posisi yang sesuai, sedangkan insertion sort menyisipkan elemen ke posisi yang tepat di antara data yang sudah terurut.